



Lab Report

Lab Report No : 04			
Lab Report Name : Local Search Algorithm: Hill Climbing search			
Course Title : Artificial Intelligence Sessional			
Course Code : CSE 422			
Name : MD Rakibul Hassan			
ID : 0822220105101024			
Level : 4	Term : 2	Section : A	Group : G1
Date of Submission : 05/07/2026	Semester : Spring	Year : 2026	

Marking Rubric

Problem Understanding & Report Clarity (3)	Implementation (4)	Results, Analysis & Presentation (3)	Total (10)

Key Learnings:

- I learned to represent graphs as nested dictionaries in Python, mapping cities to their neighbors and associated travel costs.
- I learned that hill climbing may not find the optimal path, as it doesn't explore alternative routes once a direction is chosen.
- I learned the importance of defining a proper stopping condition, such as reaching the goal or finding no neighboring node with a better heuristic value.
- I learned hill climbing is memory-efficient because it only tracks the current state, discarding previously visited nodes or paths.
- I learned hill climbing is a simple and fast optimization algorithm for many problems, but its effectiveness hinges on the heuristic's quality and the search space's structure.

Code Implementation & Sample Input/Output:

```
#Local Search Algorithms: Hill Climbing Search (Romania Road Map)
# Graph: city -> neighbors with distance
graph = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Pitesti': 97, 'Craiova': 146},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Bucharest': 101, 'Craiova': 138},
    'Craiova': {'Rimnicu Vilcea': 146, 'Pitesti': 138, 'Drobeta': 120},
    'Drobeta': {'Craiova': 120, 'Mehadia': 75},
    'Mehadia': {'Drobeta': 75, 'Lugoj': 70},
    'Lugoj': {'Mehadia': 70, 'Timisoara': 111},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}
```

Figure No: 01

```

# Straight-Line Distance Heuristic to Bucharest
heuristic = {
    'Arad': 366, 'Bucharest': 0, 'Craiova': 160, 'Drobeta': 242, 'Eforie': 161,
    'Fagaras': 176, 'Giurgiu': 77, 'Hirsova': 151, 'Iasi': 226, 'Lugoj': 244,
    'Mehadia': 241, 'Neamt': 234, 'Oradea': 380, 'Pitesti': 100,
    'Rimnicu Vilcea': 193, 'Sibiu': 253, 'Timisoara': 329,
    'Urziceni': 80, 'Vaslui': 199, 'Zerind': 374
}

# Hill Climbing Algorithm
def hill_climbing(graph, heuristic, start, goal):
    current = start # Start from the initial city
    path = [current] # Initialize path with starting city

    # Continue searching until we reach the goal
    while current != goal:
        neighbors = graph[current].keys() # Get all neighboring cities
        if not neighbors: # If no neighbors, break out of loop
            break

        # Choose the neighbor with the lowest heuristic (best improvement)
        next_city = min(neighbors, key=lambda city: heuristic[city])

        # If no improvement, stop (local maximum/plateau)
        if heuristic[next_city] >= heuristic[current]:
            break

    current = next_city # Move to the best neighbor
    path.append(current) # Add the new city to our path

    return path # Return the path found

# Solve from Arad to Bucharest
path = hill_climbing(graph, heuristic, 'Arad', 'Bucharest')

print("Path taken by Hill Climbing Search:")
print("→ ".join(path)) # Display the path as a string with arrows

```

Figure No: 02

```

# If no improvement, stop (local maximum/plateau)
if heuristic[next_city] >= heuristic[current]:
    break

current = next_city # Move to the best neighbor
path.append(current) # Add the new city to our path

return path # Return the path found

# Solve from Arad to Bucharest
path = hill_climbing(graph, heuristic, 'Arad', 'Bucharest')

print("Path taken by Hill Climbing Search:")
print("→ ".join(path)) # Display the path as a string with arrows

```

Path taken by Hill Climbing Search:
Arad → Sibiu → Fagaras → Bucharest

Figure No: 03

Result Analysis

- The **hill climbing algorithm** successfully found a path from Arad to Bucharest by consistently selecting the neighbor with the **lowest heuristic value**.
- This approach is **fast and easy to implement**, although it **does not guarantee the shortest or optimal path**.
- Its success in this instance was due to **continuously decreasing heuristic values towards the goal**.
- **Path Found:**
Arad → Sibiu → Fagaras → Bucharest